

Dev Containers

Tu propio entorno de trabajo dentro de un contenedor

Por qué un Dev Container

Instalar Java, Node o cualquier herramienta del curso directamente en tu ordenador tiene los mismos problemas que con Docker: una versión distinta a la de tu compañero, otro sistema operativo, media mañana perdida configurando un portátil nuevo.

Hasta ahora, con Docker

Metías un servicio dentro de un contenedor, como una base de datos, y tu aplicación se conectaba a él.

Con un Dev Container

Metes dentro tu propio entorno de programación, y tu editor se conecta ahí para trabajar como si fuera tu ordenador.

La misma idea que ya conoces de Docker, aplicada a tu entorno de trabajo.

La parte que cuesta entender la primera vez

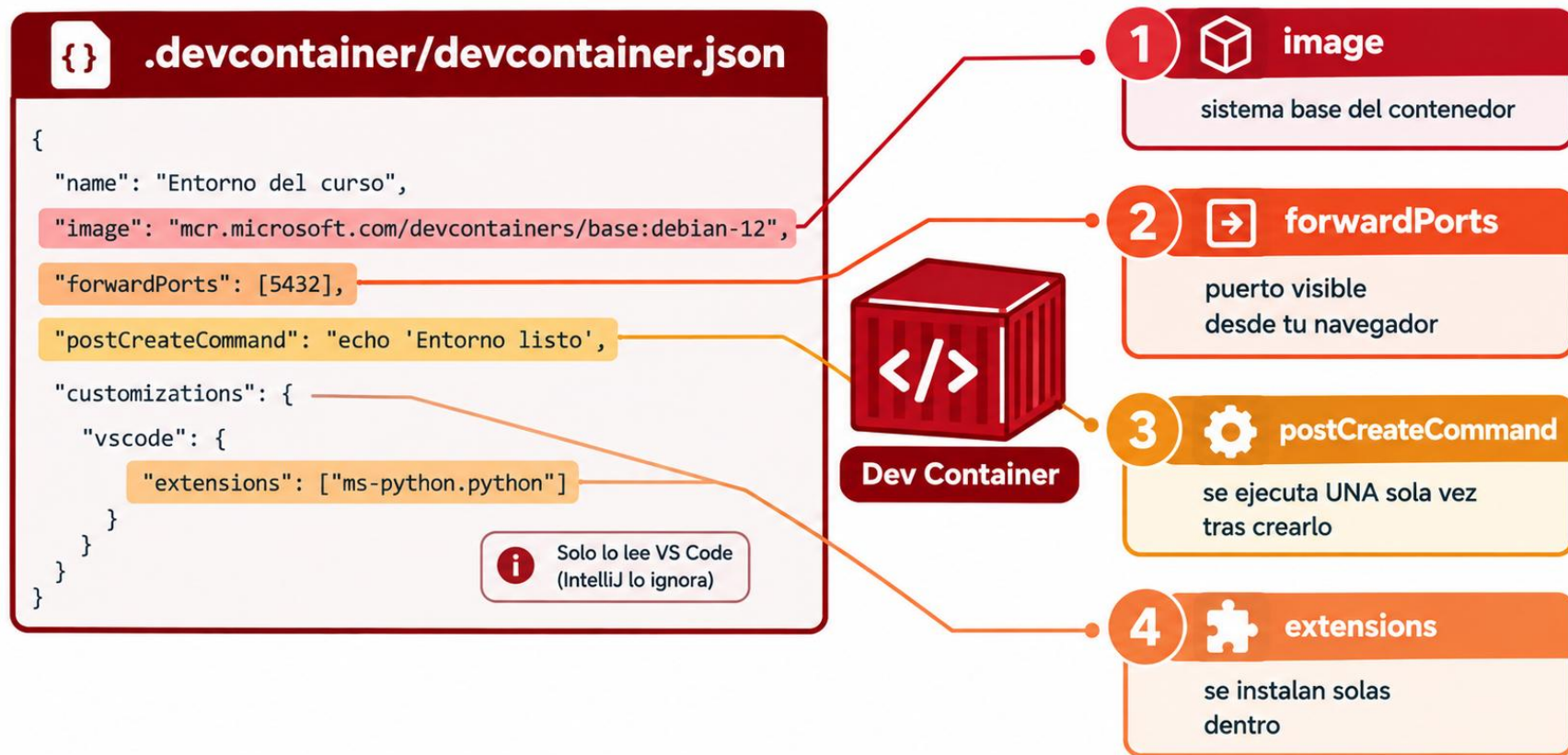
“¿Cómo que mi editor está dentro de un contenedor, si lo sigo viendo en mi propia pantalla?” La ventana se queda en tu ordenador; el trabajo de verdad corre dentro del contenedor.



Qué ocurre paso a paso

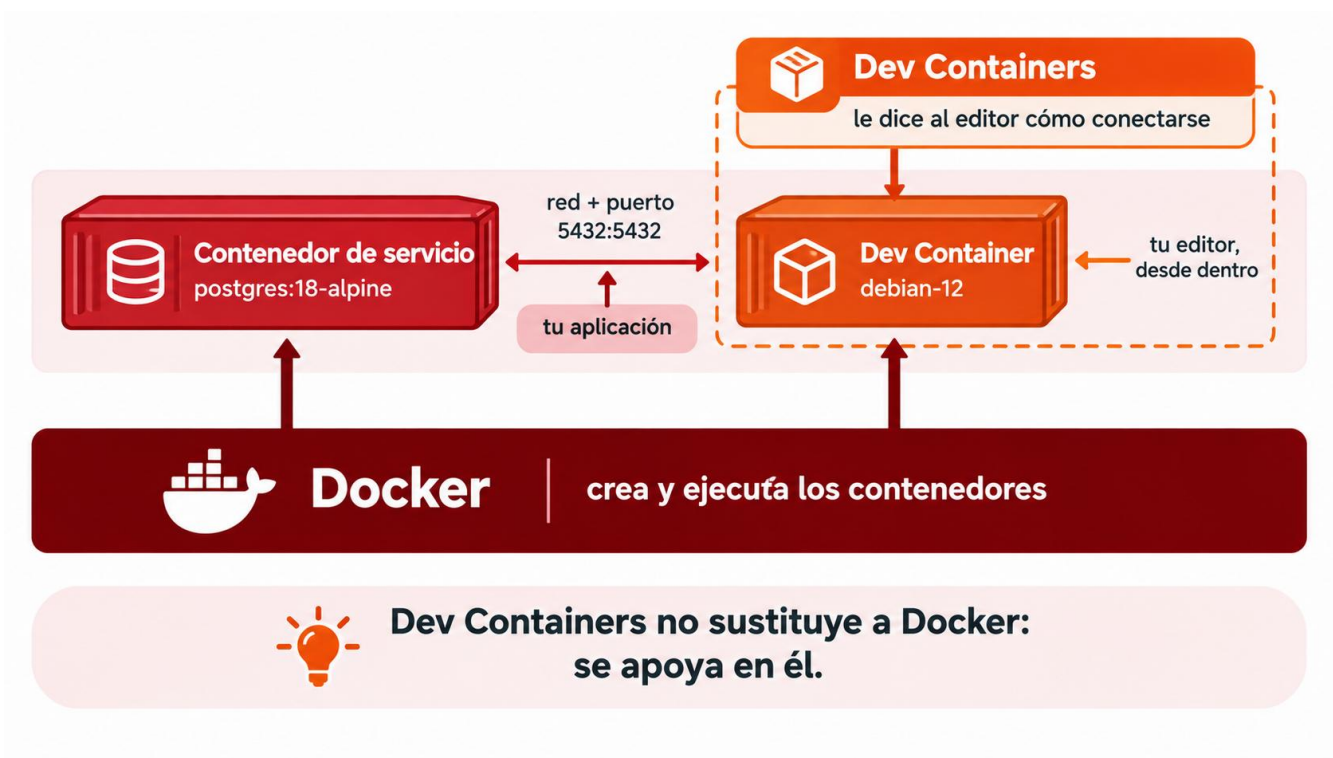


El fichero que lo describe todo



En vez de `image`, puede usar `dockerComposeFile` y `service`: el mismo `docker-compose.yml` que levanta tu base de datos puede levantar también el contenedor donde vive tu editor.

La relación con Docker, en una frase



	Contenedor de servicio	Dev Container
¿Qué hace?	Ejecuta un programa al que tu código se conecta desde fuera	Aloja tu propio entorno de programación
¿Quién se conecta?	Tu aplicación, por red y puerto	Tu editor, directamente desde dentro
Ejemplo de imagen	postgres:18-alpine	mcr.microsoft.com/devcontainers/base:debian-12

Ideas clave

1

Un Dev Container usa Docker para alojar tu propio entorno de programación, no un servicio.

2

Tu editor se conecta dentro, guiado por un fichero `.devcontainer/devcontainer.json`.

3

La ventana del editor no cambia: lo que cambia es dónde corren la terminal, el depurador y el autocompletado.

4

El `devcontainer.json` puede partir de una image suelta o de un servicio de un `docker-compose.yml`.